

Self-Healing Data Pipeline

Demo Runbook — Sprint Review Presentation

Presenter: Christian Abdel-Nour, Solutions Architect

Audience: ETS Development Team (Sean, Shai, Zhijing, Ken)

Date: July 28, 2026

Duration: ~15 minutes | Repo: github.com/sfc-gh-cabdelnour/selfhealing-pipeline

1. Demo Narrative

This demo shows a Snowflake-native self-healing pipeline that automatically detects when upstream data sources change their schema, diagnoses the root cause using Cortex AI, regenerates downstream model SQL, and validates the fix against a zero-copy clone of production — all before a human reviews and merges.

Key Message for the Audience

"When a source system drops, adds, or changes a column, your pipeline doesn't break silently at 2am. It catches the change, explains why it happened, proposes a fix, tests it, and asks a human to approve — all within Snowflake."

Design Rationale: The human-in-the-loop gate exists because LLMs can regenerate SQL correctly for simple pass-through models (SILVER), but cannot infer aggregation intent in GOLD models. The demo deliberately shows a CI_FAILED result to prove why the gate matters.

2. Architecture Overview

```
Source Change (DDL: ALTER TABLE ADD COLUMN)
|
v
[1] SCHEMA_DRIFT_DETECTOR ---- compares INFORMATION_SCHEMA vs SCHEMA_REGISTRY
|
v
[2] HALT GATE ----- blocks pipeline if PENDING events exist
|
v
[3] FN_DIAGNOSE_FAILURE ----- Cortex AI (claude-4-sonnet) root cause analysis
|
v
[4] VW_ARTIFACT_LINEAGE ----- recursive CTE: which models are affected?
|
v
[5] SP_GENERATE_ARTIFACT_CODE  Cortex LLM (llama3.1-70b) regenerates SQL
|
v
[6] SP_VALIDATE_GENERATED_CODE zero-copy clone + execute = PASS/FAIL
|
v
[7] GitHub PR (EAI) ----- opens PR with diagnosis + code + test results
|
```

3. Pre-Demo Checklist

Before presenting, verify:

1. Snowflake session active: `USE ROLE ACCOUNTADMIN; USE WAREHOUSE SELFHEALING_WH;`
2. Database exists: `SHOW DATABASES LIKE 'SELFHEALING_PROD';`
3. Schema registry is populated: `SELECT COUNT(*) FROM SELFHEALING_PROD.CONFIG.SCHEMA_REGISTRY;` (should be ~15-19 rows)
4. No pending events: `SELECT COUNT(*) FROM SELFHEALING_PROD.CONFIG.SCHEMA_CHANGE_EVENTS WHERE PIPELINE_STATUS = 'PENDING';` (should be 0)
5. Baseline SQL exists: `SELECT COUNT(*) FROM SELFHEALING_PROD.CONFIG.GENERATED_CODE WHERE EVENT_ID = 'BASELINE';` (should be ≥ 1)

If the environment is not clean: Run the teardown + build from config scripts 01 through 08 (takes ~2 minutes). See Section 7 for the full reset procedure.

4. Demo Script — Step by Step

Step 1: Show the current healthy state 1 min

Talk track: "Here's our production pipeline — 3 BRONZE tables, SILVER views, and 2 GOLD aggregate tables. Everything is stable."

```

-- Show the pipeline objects
SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_TYPE
FROM SELFHEALING_PROD.INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA IN ('BRONZE','SILVER','GOLD')
ORDER BY TABLE_SCHEMA, TABLE_NAME;

-- Show the schema registry baseline
SELECT TABLE_NAME, COLUMN_NAME, DATA_TYPE
FROM SELFHEALING_PROD.CONFIG.SCHEMA_REGISTRY
WHERE TABLE_NAME = 'ORDERS'
ORDER BY ORDINAL_POSITION;

-- Show lineage graph
SELECT * FROM SELFHEALING_PROD.CONFIG.VW_ARTIFACT_LINEAGE;

```

Design Rationale — Schema Registry: We maintain a "known-good" baseline of what the BRONZE schema looked like when everything was working. This avoids diffing against stale metadata or external catalogs. The registry advances only after a human merges the fix.

Step 2: Simulate an upstream schema change 30 sec

Talk track: "A source system just added a new column — maybe a vendor added a discount code to their API, or Finance added a field to their Excel workbook."

```

-- Simulate: source system adds a column
ALTER TABLE SELFHEALING_PROD.BRONZE.ORDERS ADD COLUMN DISCOUNT_CODE VARCHAR;

-- Verify it's there
DESCRIBE TABLE SELFHEALING_PROD.BRONZE.ORDERS;

```

Design Rationale — Why ALTER TABLE and not Openflow CDC: In production, this would happen automatically via Openflow connectors detecting source schema changes. For demo purposes, we simulate with DDL. The pipeline is source-agnostic — it only cares about the diff between INFORMATION_SCHEMA and the registry.

Step 3: Run the drift detector 1 min

Talk track: "The detector runs every 15 minutes on a scheduled task. Let's trigger it manually."

```
USE SCHEMA SELFHEALING_PROD.CONFIG;  
CALL SP_SCHEMA_DRIFT_DETECTOR();  
  
-- See the detected event  
SELECT EVENT_ID, CHANGE_TYPE, TABLE_NAME, COLUMN_NAME, PIPELINE_STATUS  
FROM SCHEMA_CHANGE_EVENTS;
```

Expected output: `NEW_COLUMN | ORDERS | DISCOUNT_CODE | PENDING`

Design Rationale — Idempotent Detection: The detector only inserts events that don't already have a PENDING/CODE_GENERATED/PR_OPEN entry for the same table+column+type. This means re-running it (e.g., if the task fires twice) never creates duplicates.

Step 4: Demonstrate the halt gate 30 sec

Talk track: "Before any pipeline refresh can proceed, we check the gate. It's blocked."

```
CALL SP_HALT_GATE();
```

Expected: `HALTED – 0 critical, 1 pending`

Design Rationale — Halt Gate: This is Shai's explicit ask: "detect schema drift in real time, halt ingestion." The gate ensures no stale or broken data propagates downstream while a schema change is unresolved. It checks both schema drift events AND data quality findings — either can block the pipeline.

Step 5: Run code generation 2-3 min

Talk track: "Now we ask Cortex AI to regenerate the impacted models. First it walks the lineage graph to find what's downstream, then it regenerates each model's SQL."

```

-- Get the event ID
SET eid = (SELECT EVENT_ID FROM SCHEMA_CHANGE_EVENTS WHERE PIPELINE_STATUS='PENDING' LIMIT 1);

-- Show what will be impacted (lineage traversal)
SELECT * FROM VW_ARTIFACT_LINEAGE WHERE CHANGED_TABLE = 'ORDERS';

-- Generate code (calls Cortex llama3.1-70b)
CALL SP_GENERATE_ARTIFACT_CODE($eid);

-- Show what was generated
SELECT ARTIFACT_NAME, ARTIFACT_SCHEMA, LEFT(GENERATED_SQL, 200) AS SQL_PREVIEW
FROM GENERATED_CODE
WHERE EVENT_ID = $eid;

```

Design Rationale — Two-Model LLM Strategy: We use `claude-4-sonnet` for diagnosis (needs strong reasoning to explain root cause) and `llama3.1-70b` for code generation (needs speed and low cost for SQL rewriting). This matches Snowflake's Cortex best practices: "Start with capable models for baseline, then optimize for cost."

Design Rationale — Recursive CTE Lineage: We use a recursive CTE over `ARTIFACT_REGISTRY` rather than parsing dbt manifest JSON. This keeps the entire system SQL-native with zero external dependencies. The lineage traversal finds ALL downstream models — not just direct dependents — so a BRONZE change correctly propagates through SILVER to GOLD.

Step 6: Validate against a zero-copy clone 1-2 min

Talk track: "Before we trust the generated code, we test it. We create a zero-copy clone of production (zero cost, instant), execute the generated SQL against it, and record pass/fail per model."

```

CALL SP_VALIDATE_GENERATED_CODE($eid);

-- Check results
SELECT ARTIFACT_NAME, TEST_STATUS, LEFT(TEST_ERROR, 100) AS ERROR
FROM GENERATED_CODE
WHERE EVENT_ID = $eid;

-- Check event final status
SELECT PIPELINE_STATUS FROM SCHEMA_CHANGE_EVENTS WHERE EVENT_ID = $eid;

```

Expected: `ORDERS_DAILY` | `FAIL` and status = `CI_FAILED`

Design Rationale — Zero-Copy Clone: Snowflake's CLONE creates an instant, full-fidelity copy of production at zero storage cost (until modified). This means we test generated code against real data structures without touching production. The clone is dropped after validation to prevent storage accumulation.

Step 7: Explain WHY it failed 2 min

Talk track: "The GOLD model failed because it's an aggregate query (GROUP BY ORDER_DATE). The LLM added DISCOUNT_CODE to the SELECT but didn't know whether to GROUP BY it or aggregate it — because that's a business decision. This is exactly why we have the human gate."

```
-- Show the generated SQL that failed
SELECT GENERATED_SQL FROM GENERATED_CODE WHERE EVENT_ID = $eid AND ARTIFACT_NAME = 'ORDERS_

-- The issue: DISCOUNT_CODE in SELECT without GROUP BY
-- A human would decide: add to GROUP BY? Or COUNT(DISTINCT DISCOUNT_CODE)?
-- Or maybe exclude from this aggregate entirely?
```

Design Rationale — Why CI_FAILED is the RIGHT demo outcome: If everything passed, the audience might think "why do we need humans?" The failure demonstrates that AI handles mechanical changes (pass-through columns in SILVER) perfectly, but cannot infer business intent for aggregations. The PR-first pattern ensures a human makes that decision — not the machine.

Step 8: Show the complete audit trail 1 min

Talk track: "Everything is logged. The detection, the generated code, the test results. In production, this would also include a GitHub PR with the diagnosis and proposed fix attached."

```
-- Full event lifecycle
SELECT EVENT_ID, CHANGE_TYPE, TABLE_NAME, COLUMN_NAME, PIPELINE_STATUS, DETECTED_AT
FROM SCHEMA_CHANGE_EVENTS;

-- Generated code archive
SELECT EVENT_ID, ARTIFACT_NAME, TEST_STATUS, GENERATED_AT
FROM GENERATED_CODE WHERE EVENT_ID != 'BASELINE';
```

5. Q&A Preparation

Likely Question

Answer

"What if the LLM generates wrong SQL that still compiles?"	The validation catches syntax errors, but not semantic errors. That's why we always require human review of the PR. The test proves it compiles and runs — correctness is the human's job.
"Can this auto-merge if the test passes?"	Architecturally yes — remove the human gate. But we deliberately chose not to because even passing code can have wrong business logic (e.g., a new column that should be excluded from an aggregate).
"How does this handle multiple simultaneous changes?"	Events are processed one at a time (LIMIT 1). Sequential processing avoids stale-branch conflicts. If multiple changes accumulate, they queue and are handled in order.
"What about column renames?"	A rename shows as COLUMN_DROP + NEW_COLUMN (two events, two PRs). We don't correlate them automatically — that's a known limitation documented in the README.
"Does this work with dbt?"	Yes — in the full version, generated SQL is committed to the dbt project and validated via EXECUTE DBT PROJECT against a clone. The demo uses direct SQL for simplicity.
"Can this handle data quality issues, not just schema drift?"	Yes — the PIPELINE_HEALTH_AUDIT table also captures null FK rates, duplicate surrogate keys, and orphan foreign keys. Same halt gate pattern. The demo focuses on schema drift because it's the most visual.

6. Closing Statement

Talk track: "This is a spike — a working proof of concept deployed on our sandbox in under 3 hours. The pattern is Snowflake-native (no external orchestrator, no Lambda, no Airflow), uses existing Cortex AI capabilities, and integrates with GitHub for change management. If the team wants this on the customer environment, it's an afternoon of adaptation work — the logic is identical, just pointed at different source tables."

7. Reset Procedure (Between Demos)

If you need to reset the environment to a clean state for a second demo:

```
-- Drop everything
DROP DATABASE IF EXISTS SELFHEALING_PROD;
DROP DATABASE IF EXISTS SELFHEALING_DEV;

-- Rebuild (run the full deploy sequence)
-- This takes ~2 minutes including Cortex model warmup
```


Then re-run Steps 1-8 from a fresh slate. The pipeline is fully idempotent — teardown and rebuild produce identical results every time (validated with 2 full cycles).